

Micro-processeur

Wafa Makni

Introduction

- Le microprocesseur est un circuit intégré complexe (ensemble de millions de transistors) capable d'effectuer séquentiellement et automatiquement des suites d'opérations élémentaires (programme) stockées en mémoire.

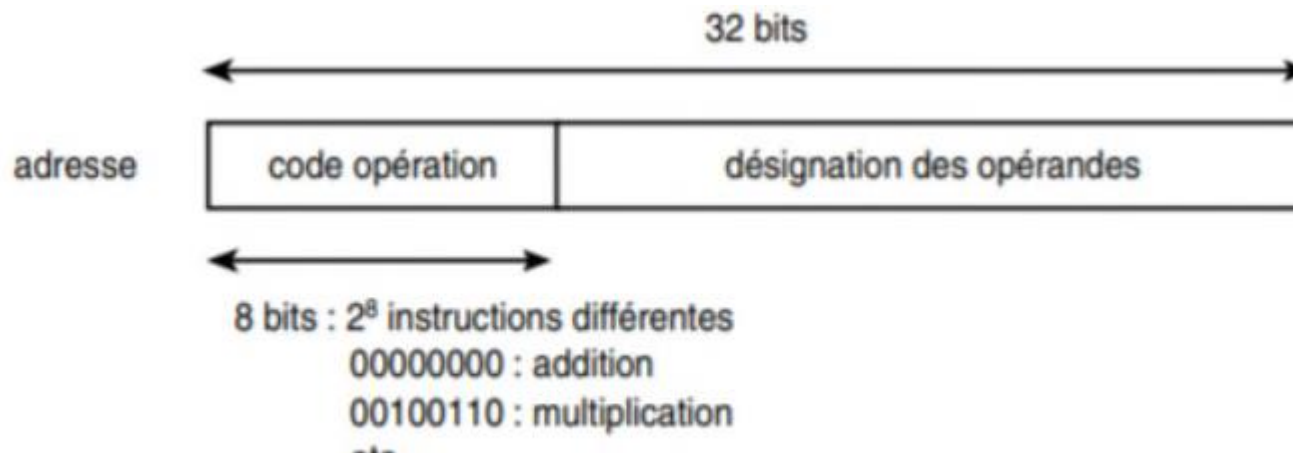
Introduction

- La vitesse du CPU est en fonction du nombre d'instructions qu'un processeur peut traiter par seconde,
- C'est ce nombre d'impulsions par seconde qui détermine la **fréquence d'horloge** du microprocesseur.
- Cette fréquence est exprimée en hertz.
- Un microprocesseur cadencé à 500 MHz envoie donc 500 millions d'impulsions/seconde.
- A chaque impulsion, le microprocesseur exécute donc une instruction (ou une partie d'une instruction).

Langage de programmation

Langage machine

- Le langage machine est le langage compris par le microprocesseur.
- Ce langage est difficile à maîtriser puisque chaque instruction est codée par une séquence propre de bits.
- Afin de faciliter la tâche du programmeur, on a créé différents langages plus ou moins évolués.



Langage de programmation

Langage assembleur

- C'est le langage le plus proche du langage machine
- Utilisant les mnémoniques qui facilite la programmation. (exemple: ***MOV AX, BX***)
- Le langage assembleur des processeurs INTEL est différent de celui des processeurs MOTOROLA
- Chaque microprocesseur peut posséder un assembleur différent.

Langage de programmation

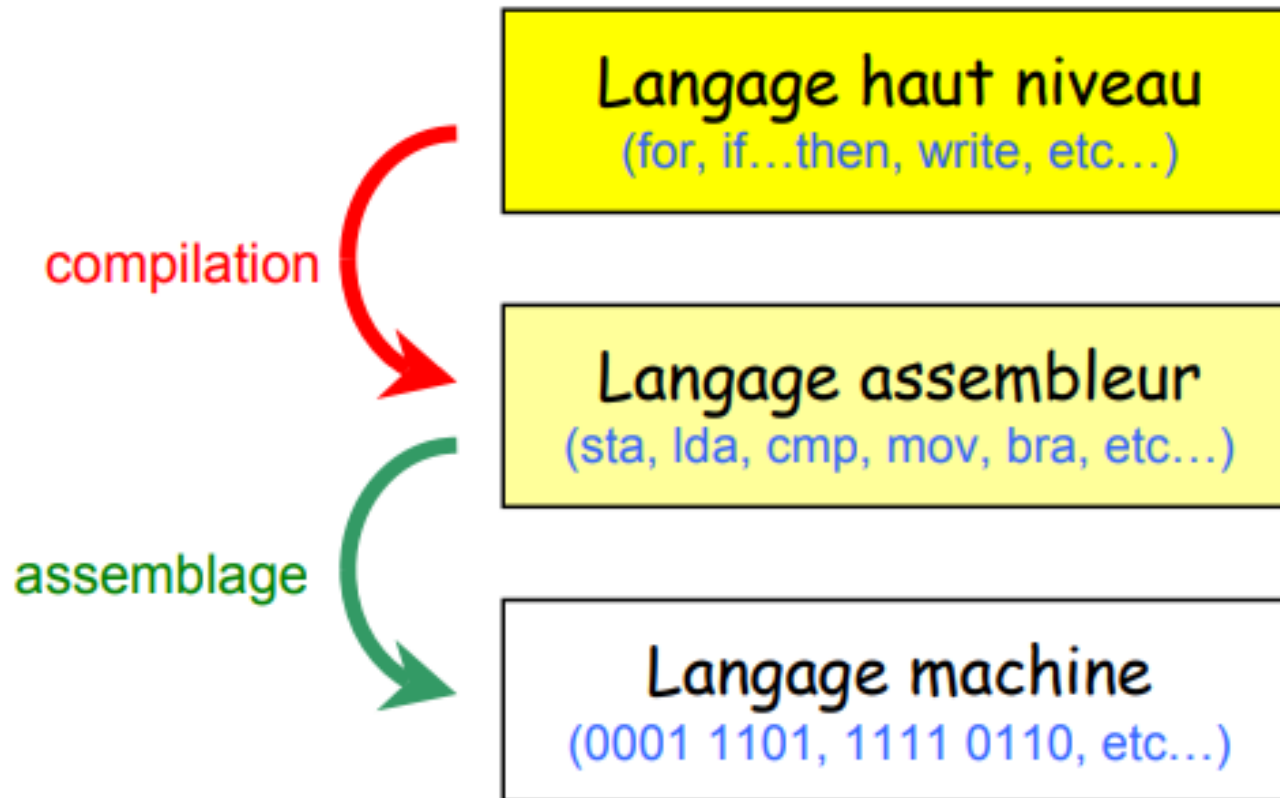
Langage haut niveau

- La difficulté de mise en œuvre du langage assembleur, et leur forte dépendance avec la machine a nécessité la conception de langages de haut niveau, plus adaptés à l'homme, et aux applications qu'il cherchait à développer (Java, Pascal, C, etc...).

Langage de programmation

- le programme en langage de haut niveau n'est donc pas compréhensible par le microprocesseur.
- Il faut le compiler pour le traduire en assembleur puis l'assembler pour le convertir en code machine compréhensible par le microprocesseur.
- Ces opérations sont réalisées à partir de logiciels spécialisés appelés compilateur et assembleur.

Langage de programmation



Langage de programmation

Exemple de programme :

Code machine (68HC11)	Assembleur (68HC11)	Langage C
@00 C6 64	LDAB #100	A=0 ;
@01 B6 00	LDAA #0	for (i=1 ; i<101 ; i++) A=A+i ;
@03 1B	ret ABA	
@04 5A	DECB	
@05 26 03	BNE ret	

Activer Win

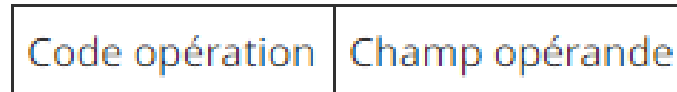
Comparaison

Langage haut niveau / Assembleur

Langage haut niveau	Langage assembleur
⊕ rapide à coder ⊕ Langage facile ⊖ Exécution lente (code non optimisé donc le compilateur va engendrer un programme beaucoup plus volumineux)	⊖ temps de programmation élevé ⊖ Langage complexe (architecture du μP, les registres...) ⊕ Exécution rapide (programme optimisé ce que le compilateur ne fait pas)

Instructions

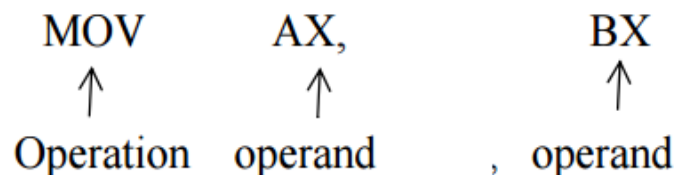
Une instruction est composée de deux champs :



- le **code opération**, représentant l'action que le processeur doit accomplir ;
- le **champ opérande**, Un opérande peut être une valeur, un registre du CPU, une donnée située en mémoire vive

Exemple:

- l'instruction assembleur « **MOV AX, BX** » va copier la donnée qui se trouve dans le registre BX dans le registre AX .



Exemple de processeur 8086

- L'Intel 8086 est un microprocesseur fabriqué par Intel à partir de 1978.
- C'est le premier processeur de la famille x86, qui est devenue l'architecture de processeur la plus répandue dans le monde.

processeur 8086

- Les Registres généraux de 16 bits: **AX, BX, CX, DX**

AX (16 bits) = AH (8 bits hauts) + AL (8 bits bas)

AL: partie basse (bits 0...7)

AH: partie haute (bits 8...15)

BX (16 bits) = BH (8 bits hauts) + BL (8 bits bas)

CX (16 bits) = CH (8 bits hauts) + CL (8 bits bas)

DX (16 bits) = DH (8 bits hauts) + DL (8 bits bas)

➔ Ces registres sont les plus utilisés en programmation 8086

processeur 8086

Types d'instructions

- Instruction de transfert
- Les instructions Arithmétiques
- Les instructions Logiques

processeur 8086

Instruction de transfert

✓ Instruction MOV

MOV destination, source

Les transferts possibles sont :

Destination	Source
Registre	Registre
Registre	Mémoire
Mémoire	Registre
Registre	Valeur immédiate
Mémoire	Valeur immédiate

processeur 8086

Instruction de transfert

✓ Instruction MOV

■ Exemple

MOV AX, BX : copier le contenu du registre Bx dans le registre Ax.

Dans ce cas, le registre Bx (source) et le registre Ax (destination)

processeur 8086

Instruction Arithmétiques

✓ Instruction ADD

ADD, destination ,source

Destination <= destination +source

Exemple:

MOV AX, 0

ADD AX, 100 ;AX = 0 + 100 = 100 ;

Résultat :

; AX = 100 (décimal) = 0064h (hexadécimal)

AH = 00h ; AL = 64h (= 100 en décimal)

processeur 8086

Instruction Arithmétiques

✓ Instruction SUB

SUB, destination ,source

Destination<= destination -source

Exemple:

SUB AX,4 $AX \leq AX - 4$

processeur 8086

Instruction Arithmétiques

✓ Instruction MUL (multiplication non signée (nombres positifs uniquement))

MUL, destination ,source

Destination <= destination X source

Multiplication 8 bits

MUL BL AX <= AL X BL

Exemple

MOV AL, 5

MOV BL, 3

MUL BL ; $AL \times BL = 5 \times 3 = 15 \rightarrow AX = 15$

processeur 8086

Instruction Arithmétiques

✓ Instruction MUL

Pour une opérande 8 bits :

- Le **multiplicande** est toujours **AL** (implicite)
- Le **multiplicateur** peut être **n'importe quel registre 8 bits** (AL, AH, BL, BH, CL, CH, DL, DH)
- Le **résultat** est toujours dans **AX**

processeur 8086

Instruction Arithmétiques

- Instruction MUL
- MUL AL ; $AL \times AL \rightarrow AX$
- MUL AH ; $AL \times AH \rightarrow AX$
- MUL BL ; $AL \times BL \rightarrow AX$
- MUL BH ; $AL \times BH \rightarrow AX$
- MUL CL ; $AL \times CL \rightarrow AX$
- MUL CH ; $AL \times CH \rightarrow AX$
- MUL DL ; $AL \times DL \rightarrow AX$
- MUL DH ; $AL \times DH \rightarrow AX$

processeur 8086

Instruction Arithmétiques

✓ Instruction MUL

MUL, destination ,source

Destination $\leftarrow$ destination X source

Multiplication 16 bits

MUL BX DX:AX $\leftarrow$ AX * BX

- **Résultat** : DX:AX (32 bits)
- Poids fort dans DX
- Poids faible dans AX

processeur 8086

Instruction Arithmétiques

Instruction MUL

Pour une multiplication **16 bits**, le **multiplicande** est **toujours AX** (implicite).

- MUL AX ; $AX \times AX \rightarrow DX:AX$
- MUL BX ; $AX \times BX \rightarrow DX:AX$
- MUL CX ; $AX \times CX \rightarrow DX:AX$

processeur 8086

Instruction Arithmétiques

✓ INC

INC AX : incrémenter le registre AX (AX++)

processeur 8086

Les instructions Logiques

✓ Instruction AND

AND, destination ,source

Destination<= destination (AND) source

Exemple:

```
MOV AX , 503H ; AX = 0000 0101 0000 0011
```

```
AND AX , 0201H ;
```

```
0000 0101 0000 0011 ;  
AND 0000 0010 0000 0001 ;  
0000 0000 0000 0001
```

processeur 8086

Les instructions Logiques

✓ Instruction OR

OR, destination ,source

Destination<= destination (OR) source

Exemple:

```
MOV AX , 503H ; AX = 0000 0101 0000 0011
OR AX , 0201H      0000 0101 0000 0011 ;
                   OR  0000 0010 0000 0001
                   0000 0111 0000 0011
```

Les modes d'adressage

- **Adressage immédiat** : lorsque le code opérande contient une donnée

Exemple : **MOV AL, 21h** ; permet de transférer 21h dans le registre AL.

- **Adressage d'un registre** : La donnée est contenue dans un registre.

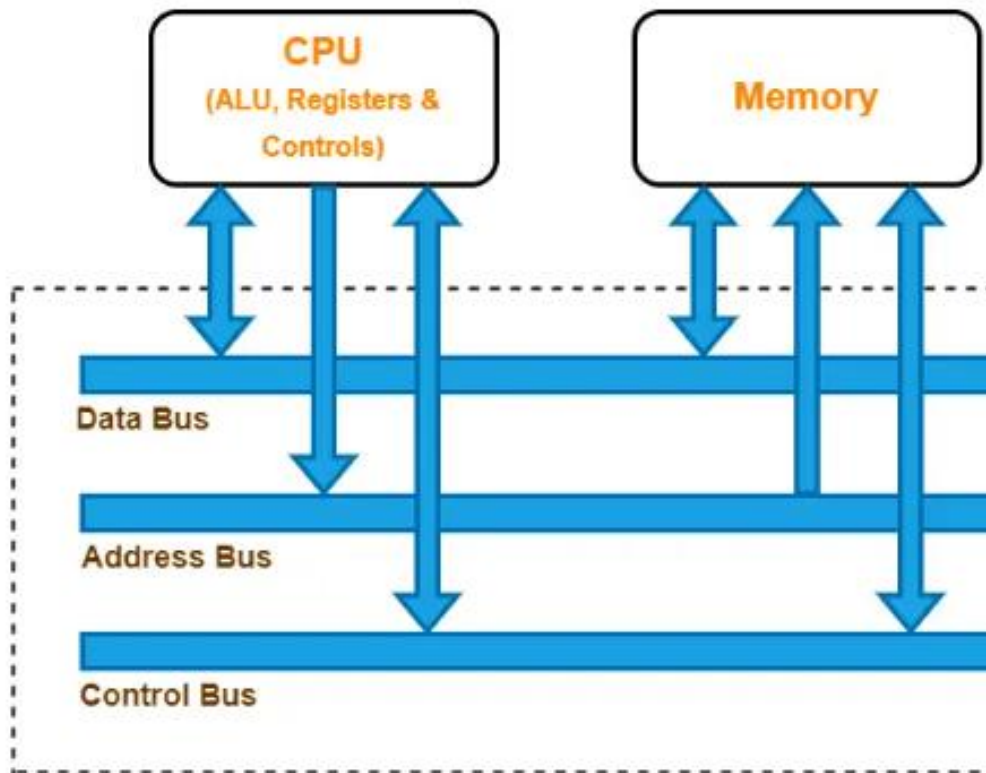
Exemple : **MOV AX, BX** : charge le contenu du registre BX dans le registre AX.

Les modes d'adressage

- **Adressage direct ou étendu** : L'instruction comporte l'adresse effective de la case mémoire où se trouve la donnée.
- Exemple : **MOV BL, [0200h]** ; transfère le contenu de la case mémoire d'adresse 0200h vers le registre BL.

Architecture de base d'un microprocesseur

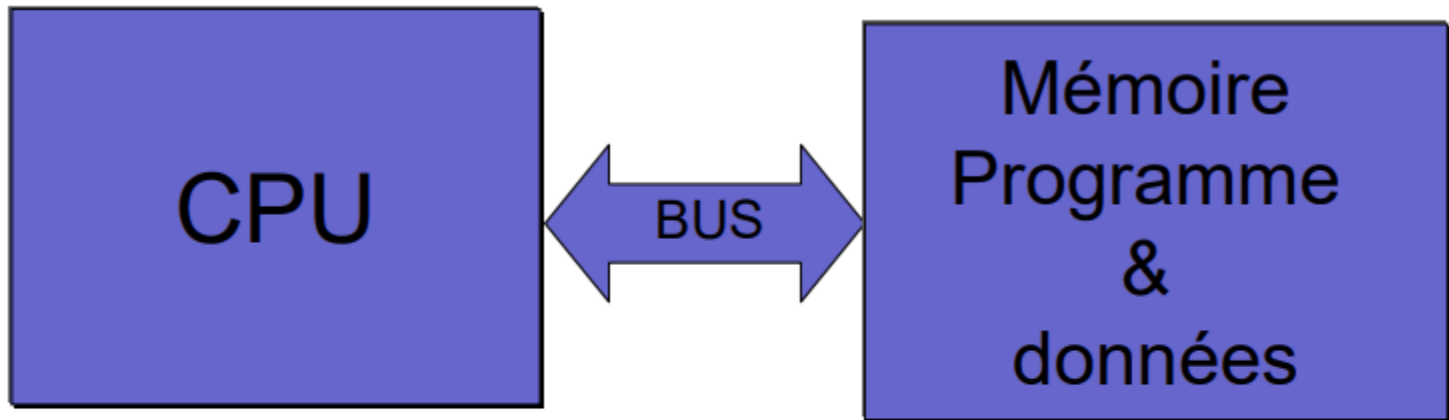
Bus



- **Bus d'adresses**
Unidirectionnel sur lequel le processeur envoie les adresses des cellules mémoire.
- **Bus de données**
est bidirectionnel puisque le processeur l'utilise pour lire et écrire en mémoire
- **Bus de commandes (Control Bus)**
est bidirectionnel:
 - Le CPU utilise l'un d'eux pour indiquer le sens des transferts sur le bus de données (lecture ou écriture).
 - l'autre sens que les mémoires signalent quand elles sont prêtes pour répondre à une commande de lecture

Architecture de base d'un microprocesseur

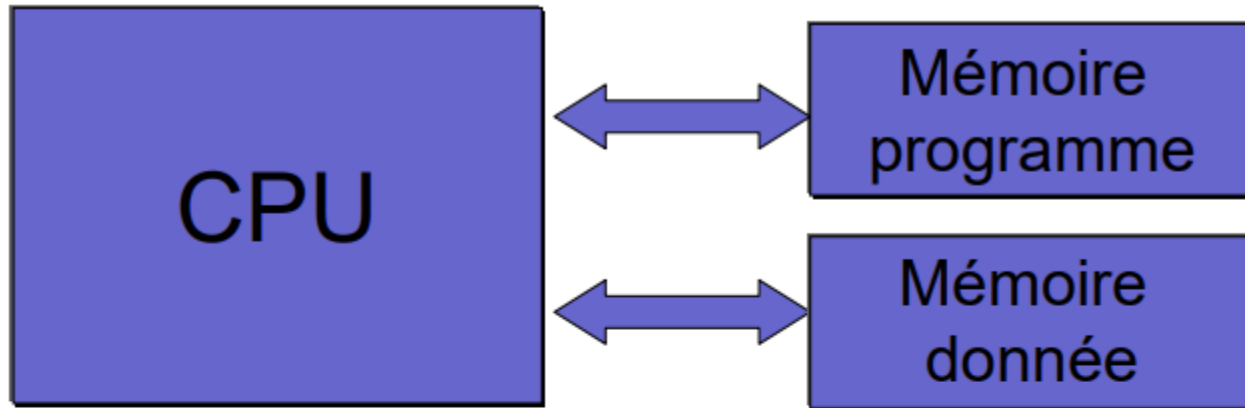
Von Neuman



- Un seul chemin d'accès à la mémoire
 - Un bus de données (programme et données),
 - Un bus d'adresse (programme et données)

Architecture de base d'un microprocesseur

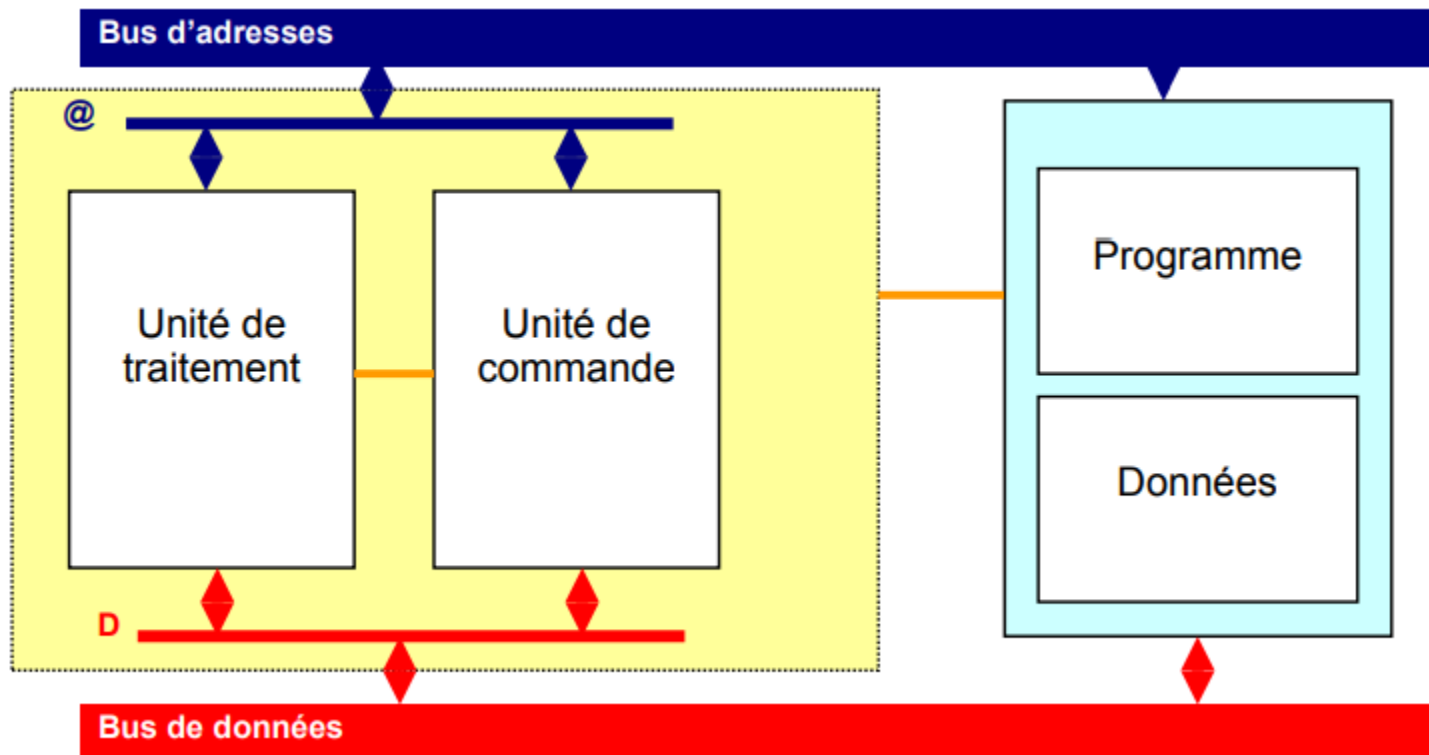
Harvard



- Séparation des mémoires programme et données
 - Un bus de données programme,
 - Un bus de données pour les données,
 - Un bus d'adresse programme,
 - Un bus d'adresse pour les données.
- Meilleure utilisation du CPU :
 - Chargement du programme et des données en parallèle

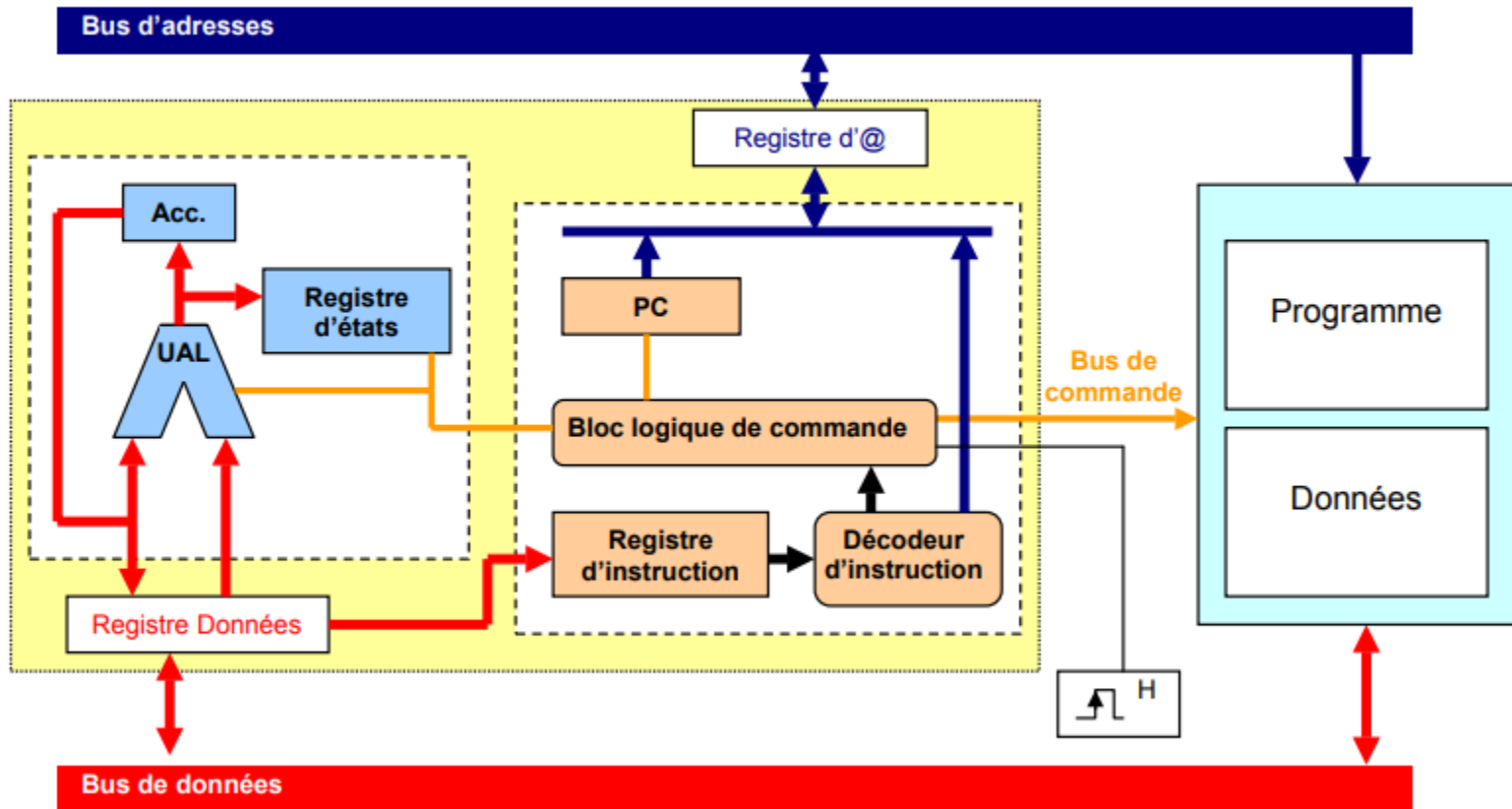
Architecture de base d'un microprocesseur

- microprocesseur est construit de deux éléments principaux :
 - Une unité de commande
 - Une unité de traitement



Architecture complète

Schéma fonctionnel



unité de commande

- l'unité de commande est responsable de la lecture en mémoire principale et du décodage des instructions elle est composée par :

❑ **Registre instruction (RI)** : contient l'instruction en cours d'exécution.

❑ **décodeur d'instruction**: détermine le type de l'opération à effectuer

unité de commande

□ compteur ordinal (CO) ou le compteur de programme (CP)

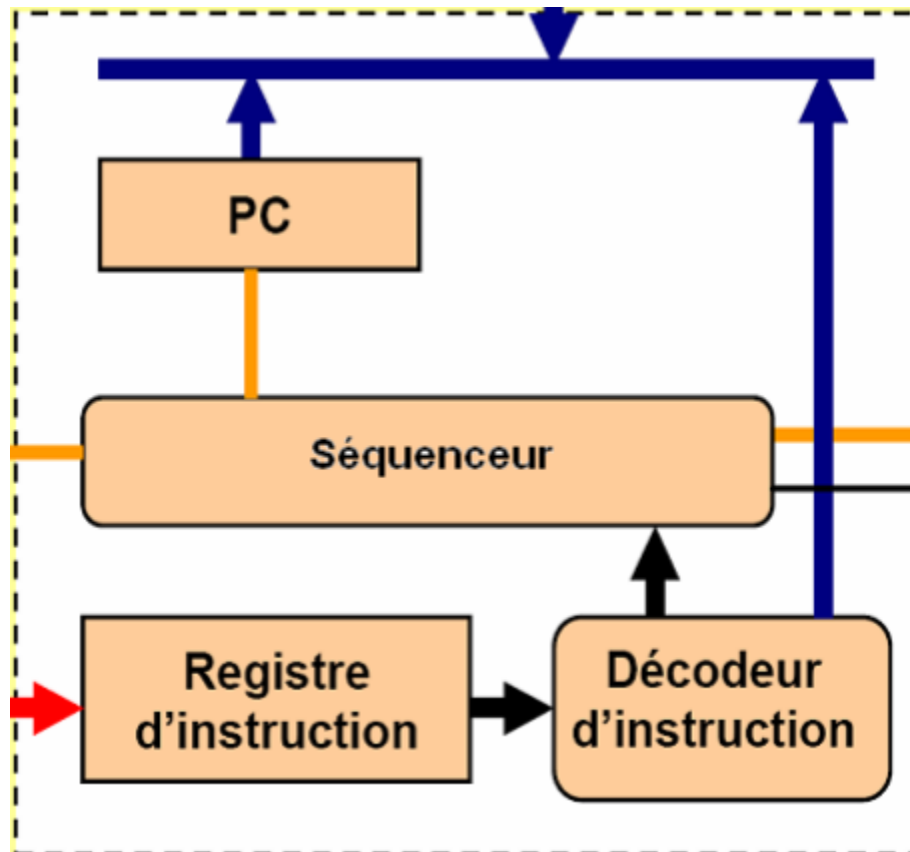
- Un registre dont le contenu est initialisé avec l'adresse de la première instruction du programme.
- Dès le lancement du programme ce compteur contient l'adresse de la prochaine instruction à exécuter:
 - ✓ soit par incrémentation automatique dans le cas où les adresses des instructions se suivent;
 - ✓ soit par chargement de l'adresse de branchement dans le cas de sauts programmés.

unité de commande

□ séquenceur

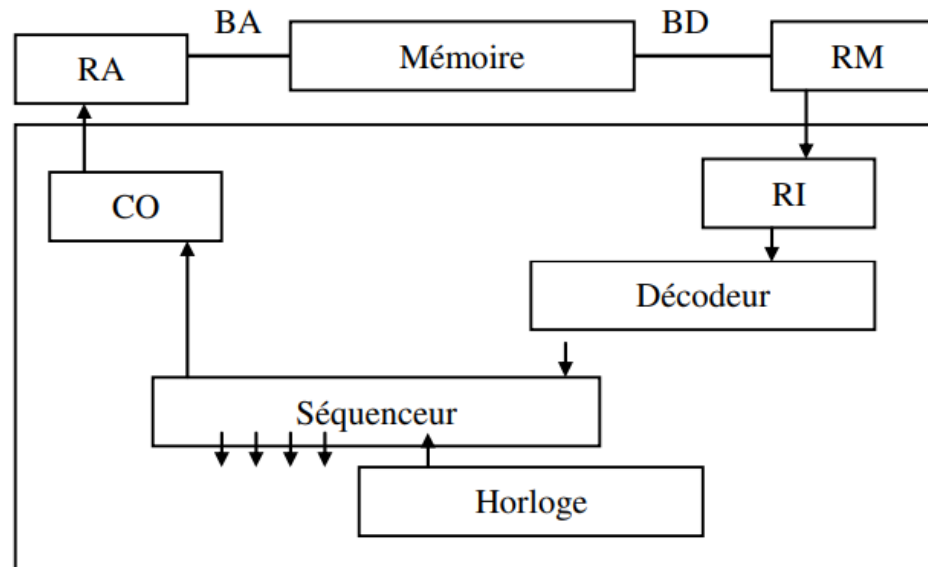
- Il organise (synchronise) l'exécution des instructions selon le rythme de l'horloge, il génère les signaux nécessaires pour exécuter une instruction.
- Le séquenceur active les bus de données, d'adresses et de contrôle au bon moment pour permettre les transferts entre les différents composants (registres, ALU, mémoire).
- Il incrémente le compteur de programme pour passer à l'instruction suivante, ou le modifie en cas de branchement ou de saut.
- Il envoie des signaux de contrôle à l'UAL (Unité Arithmétique et Logique) pour qu'ils effectuent les opérations requises.

unité de commande



unité de commande

RA :Registre d'Adresse
RM : Registre de Mot
RI : Registre d'instruction
CO : compteur Ordinal
BA :Bus d'Adresses
BD :Bus de Données



Unité de
commande

Unité de traitement

- Elle regroupe les circuits qui assurent les traitements nécessaires à l'exécution des instructions :
- **Unité Arithmétique et Logique (UAL) :**
est un circuit qui réalise les fonctions logiques (ET, OU, Comparaison, Décalage , etc...) ou arithmétique (Addition, soustraction).

Unité de traitement

- Le registre d'état:
 - composé de 8 bits (généralement).
 - Chacun de ces bits est un indicateur qui donne l'état du résultat de la dernière opération effectuée par l'UAL.
 - On les appelle indicateur d'état ou flag ou drapeaux.

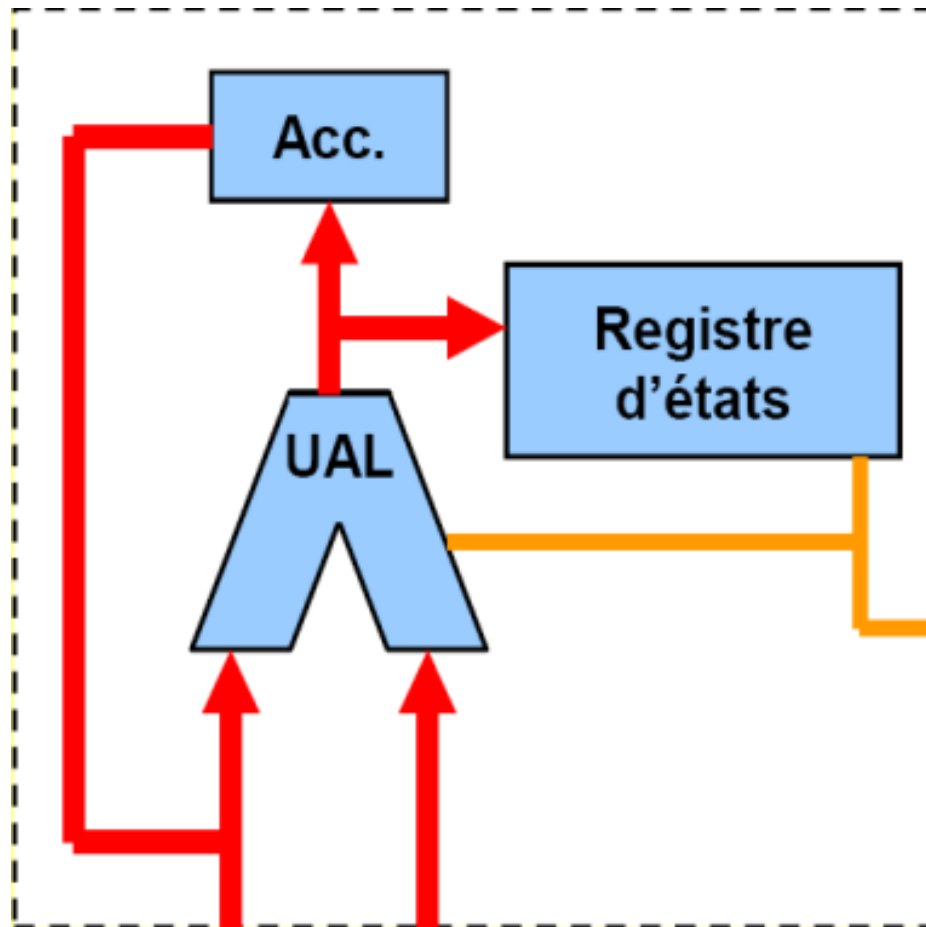
On peut citer par exemple les indicateurs de :
- retenue (carry : C)
- retenue intermédiaire (Auxiliary-Carry : AC)
- signe (Sign : S)
- débordement (overflow : OF)
- zéro (Z)
- parité (Parity : P) ☐

Unité de traitement

- L'accumulateur:

L'accumulateur est un **registre de travail** qui stocke temporairement les données pendant les opérations arithmétiques et logiques.

Unité de traitement



Traitement d'une instruction

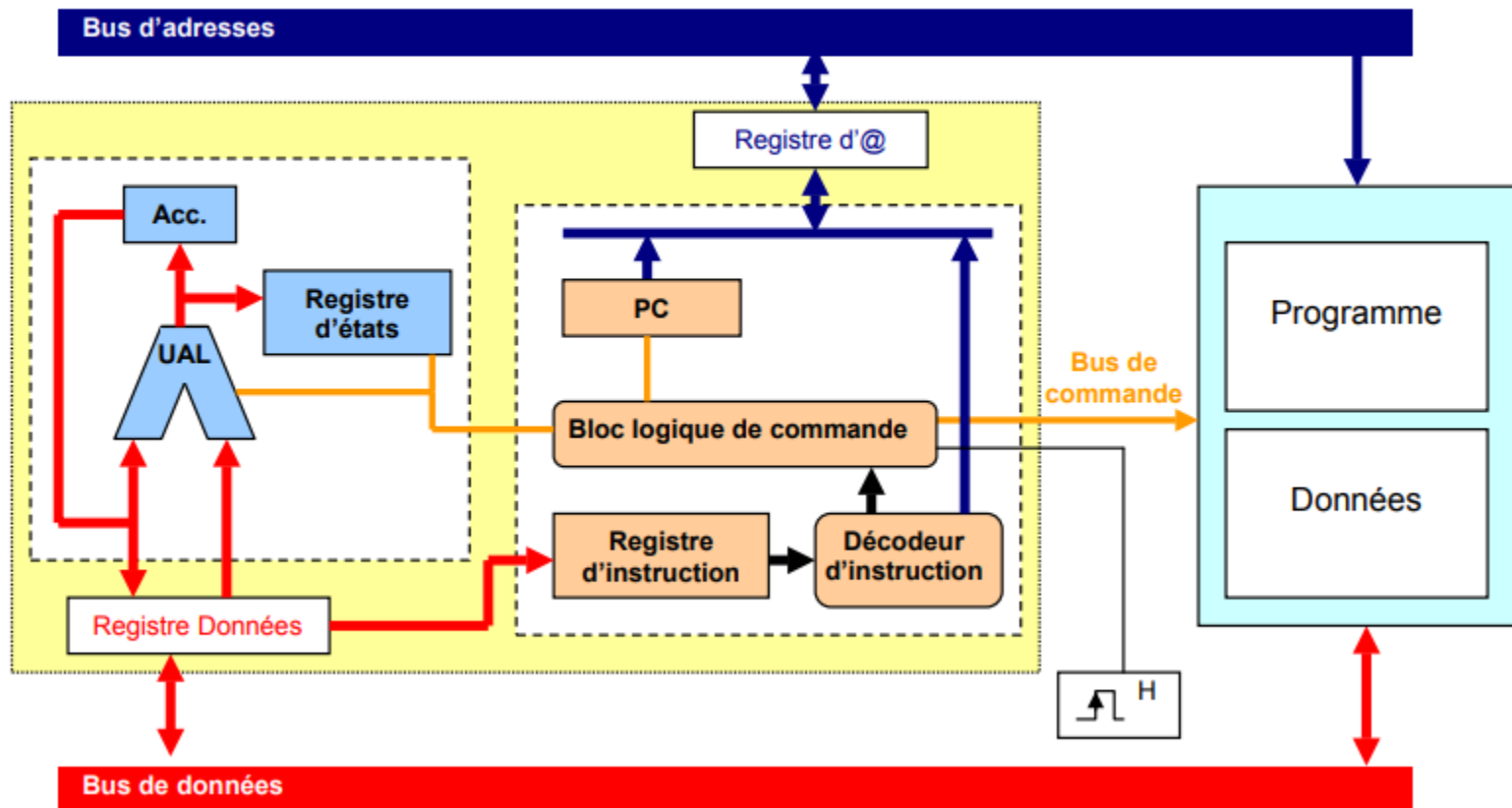
Le traitement d'une instruction par le processeur se décompose en trois étapes :

- Recherche de l'instruction
- Décodage de l'instruction
- Exécution de l'instruction

Traitement d'une instruction

Phase1: Recherche de l'instruction en mémoire

Schéma fonctionnel



Traitement d'une instruction

Phase1: Recherche de l'instruction en mémoire

1. Transfert de l'adresse de la nouvelle instruction du **CO** vers le **RA** (registre adresse de la mémoire)
2. Une impulsion de lecture, générée par l'unité de commande provoque le transfert de l'instruction recherché vers le **RM**(registre Donnée)
3. Transfert de l'instruction dans le **RI**
(instruction=code opération +adresse opérande)

Traitement d'une instruction

Phase2:Décodage

- Le code opération est transmis au décodeur qui détermine le type d'opération demandée et le transmet au séquenceur (activer le circuit).
- Connaître la ou les adresses effectives des opérandes nécessaires à l'exécution de l'instruction.
- Le CO est incrémenté

Traitement d'une instruction

Phase3:Exécution de l'instruction

1. Le séquenceur commence à envoyer les signaux de commande vers la mémoire pour lire l'opérande à l'adresse déjà stockée dans le **RA** et le faire parvenir dans de **RM**.
2. Transfert du contenu du **RM** vers UAL
3. Les drapeaux sont positionnés (registre d'état).

Pipeline

- Technique utilisée pour optimiser le temps d'exécution d'un processus répétitif.
- Avec un pipeline, le processeur peut commencer à exécuter une nouvelle instruction sans attendre que la précédente soit terminée
- Plusieurs instructions se trouvent simultanément en cours d'exécution au cœur du micro-processeur.
- Afin de mettre en œuvre un pipeline, la première tâche est de découper l'exécution des instructions en plusieurs *étapes*
- augmenter le nombre d'instructions exécutées par unité de temps.

Pipeline

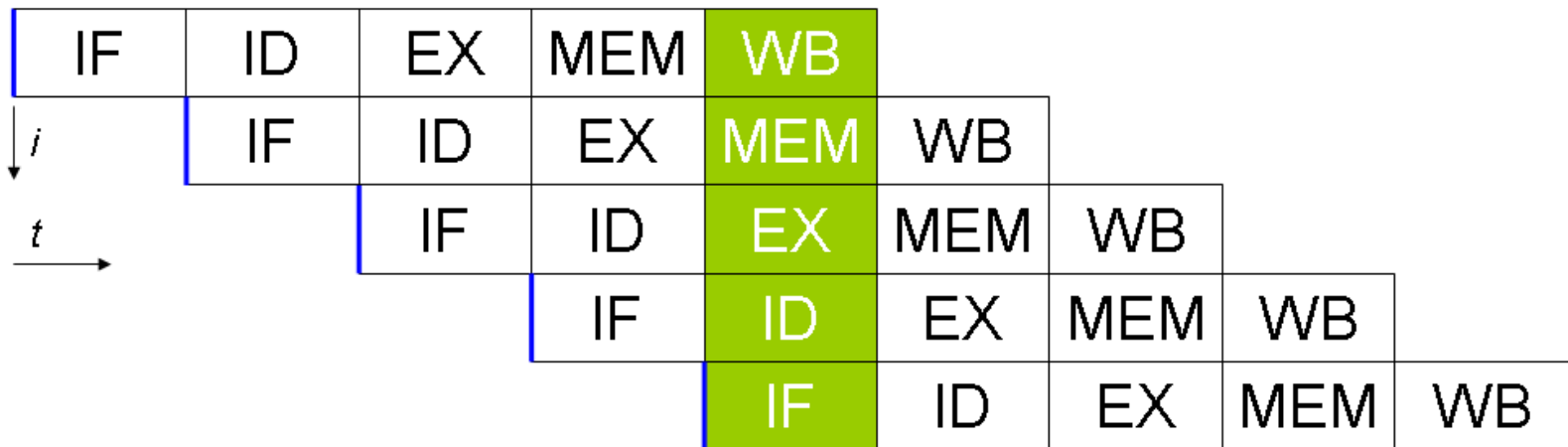
■ Exemple sans pipeline

- Dans un micro-processeur sans pipeline, les instructions sont exécutées les unes après les autres
- En supposant que chaque étape met 1 cycle d'horloge, il faut donc 5 cycles pour exécuter une instruction, 15 cycles horloges pour 3 instructions.



Pipeline

- Pipeline (exemple pipeline 5 étages)



Il faut 9 cycles pour exécuter 5 instructions

Au 5^e cycle, tous les étages sont en cours d'exécution.

Pipeline

- Pipeline
- Exemple: 5 étapes sont nécessaires pour traiter une instruction
- IF (Instruction Fetch) charge l'instruction à exécuter dans le pipeline.
- ID (Instruction Decode) décode l'instruction
- EX (Execute) exécute l'instruction (par la ou les unités arithmétiques et logiques).
- MEM (Memory), accès mémoire en lecture ou écriture ;
- WB (Write Back) écriture du résultat d'une lecture ou d'un calcul dans les registres.

Pipeline

- Temps d'exécution sans pipeline:

$$n * k * T_c$$

n: nombre d'instructions

K: nombre d'étages

T_c: temps (durée) de cycle

Pipeline

- Temps d'exécution avec pipeline:

le nombre de cycles * temps (durée) de cycle

Temps d'exécution avec pipeline = $(n+k-1) * T_c$

n: nombre d'instructions

K: nombre d'étages du pipeline

Exemple:

Nombre de cycles = $6 + (100 - 1) = 105$ cycles

$T_{\text{cycle}} = 1 / 3 \text{ GHz} \approx 0,333 \text{ ns}$

Temps_exécution = $105 \times 0,333 \text{ ns} \approx 35 \text{ ns}$

Pipeline

Le Gain

Le gain mesure **combien de fois plus rapide** est l'exécution avec pipeline :

$$\text{Gain} = \text{Temps_sans_pipeline} / \text{Temps_avec_pipeline}$$

$$\text{Gain} = (n \times k) / (k + n - 1)$$

Exercice N1

- Pour traiter une instruction le microprocesseur divise l'instruction en 5 étages: (**LE** : Lecture de l'instruction en mémoire, **DE** : Décodage de l'instruction, **CH** : Chargement des registres sources dans l'unité de calcul, **EX** : Exécution du calcul, **ENR** : Enregistrement du résultat dans le registre destination).
Le temps de cycle de ce processeur est de 0.5ns.
- On considère le fragment du code suivant :
- `ADD R1, R2, R3` $R1 \leftarrow R2 + R3$
- `MUL R4, R5, 5` $R4 \leftarrow R5 \times 5$
- `DIV R8, R9, R10` $R8 \leftarrow R9 / R10$
- `SUB R11, R12, 7` $R11 \leftarrow R12 - 7$

Exercice N1

1. Dessiner le diagramme d'exécution pour le fragment du code suivant dans le cas séquentiel (non pipeliné)
2. Donner le temps d'exécution du non-pipeline pour le fragment du code suivant
- 3 Dessiner le diagramme d'exécution pour le fragment de code suivant utilisant le pipeline
- 4 Donner le temps d'exécution du pipeline pour le fragment du code suivant
- 5 Quelle est l'accélération ?
- 6 Supposant qu'on a 1000 instructions, en utilisant l'approximation ($n \gg k$) quelle sera l'accélération

Les Aléas

- Les **aléas** sont des **situations problématiques** qui empêchent l'exécution fluide du pipeline et réduisent ses performances.
- Ils créent des **bulles (cycle d'horloge perdu dans le pipeline)** dans le pipeline.
- Aléas de données
- Aléas Structurels

Aléa structurel

- Quand deux instructions ont besoin d'utiliser la même ressource du processeur.
- Exemple:

1	2	3	4	5	6
F	DC	EX	M	WB	
	F	DC	EX	M	WB
		F	DC	EX	M
			F	DC	EX

Aléas de données

Une instruction dépend du résultat d'une instruction précédente qui n'est pas encore terminée.

	1	2	3	4	5	6	7	8	9	10	11	12
add r6, r5, r2	F	DC	EX	M	WB							
add r3, r1, r6		F	DC			EX	M	WB				
add r5, r1, r2			F			DC	EX	M	WB			

Exercice N2

- Pour traiter une instruction le microprocesseur divise l'instruction en 5 étages:
(**LE** : Lecture de l'instruction en mémoire, **DE** : Décodage de l'instruction, **CH** : Chargement des registres sources dans l'unité de calcul, **EX** : Exécution du calcul, **ENR** : Enregistrement du résultat dans le registre destination).

Le temps de cycle de ce processeur est de 0.5ns.

- On considère le fragment du code suivant :
- `ADD R1, R2, R3` $R1 \leftarrow R2 + R3$
- `MUL R4, R1, 5` $R4 \leftarrow R1 \times 5$
- `DIV R8, R9, R10` $R8 \leftarrow R9 / R10$
- `SUB R11, R12, 7` $R11 \leftarrow R12 - 7$

Exercice N2

1. Identifier et expliquer brièvement l'aléa présent dans les instructions suivantes.
2. Dessiner le diagramme d'exécution pour le fragment de code suivant utilisant le pipeline
3. D'après le diagramme, combien de cycle faut-il pour exécuter ce programme en pipeline

Pipeline

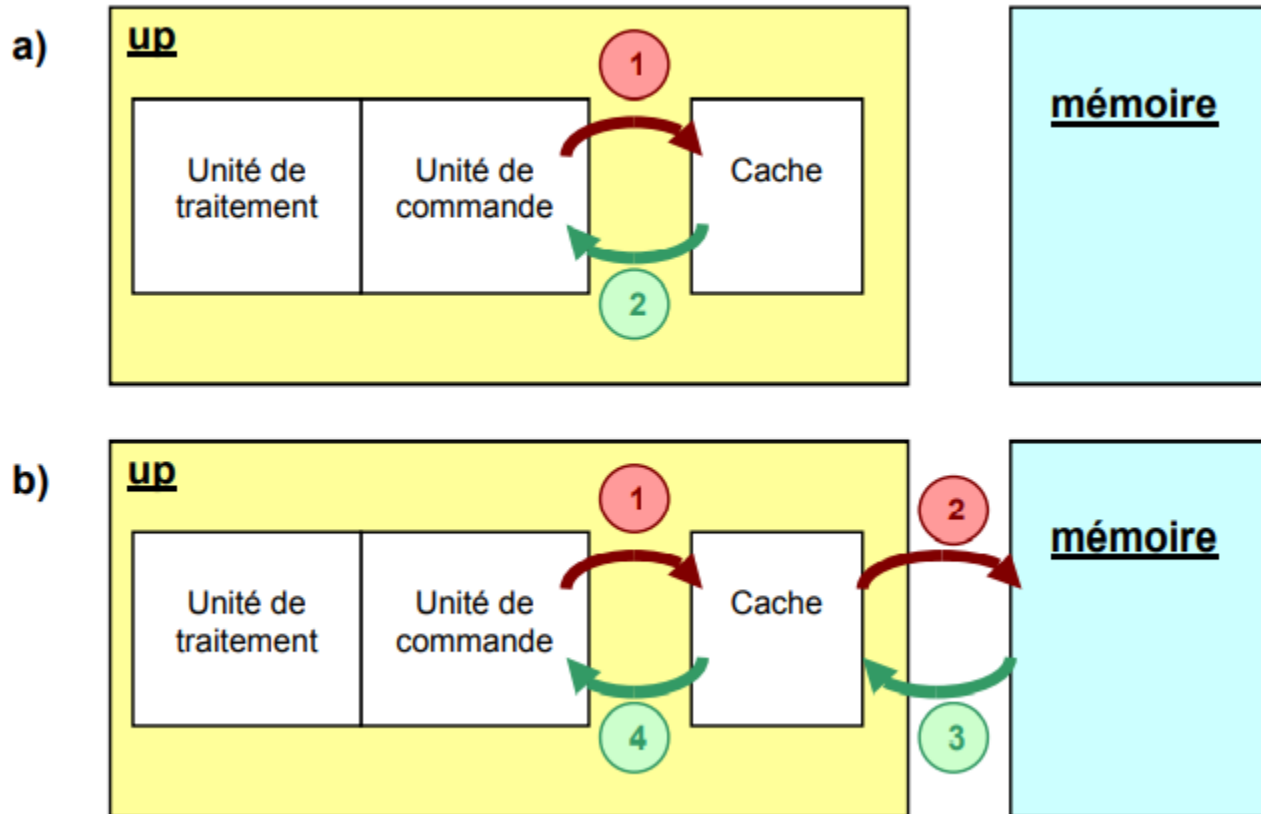
Exercice N3

Considérons un pipeline à **6 étages** de temps : *50 ns, 50 ns, 60 ns, 60 ns, 50 ns et 50 ns*. Dans un premier temps, on n'utilise pas le pipeline. Combien de temps prennent l'exécution de 100 instructions ?

La mémoire cache

- Les mots de mémoire centrale les plus fréquemment utilisés sont placés dans la mémoire cache.
- Le processeur cherche d'abord la donnée ou l'instruction dans la mémoire cache, en cas d'échec le processeur fait accès à la mémoire centrale et place l'information dans le cache.
- Ultérieurement si cette information est demandée elle sera obtenue plus rapidement.

principe de cache



principe de cache

- Si la donnée ou l'instruction requise est présente dans le cache, elle est alors envoyée directement au microprocesseur.

⇒ On parle de **succès de cache**. (a) ☐

si la donnée ou l'instruction n'est pas dans le cache, le contrôleur de cache envoie alors une requête à la mémoire principale.

Une fois l'information récupérée, il la renvoie au microprocesseur tout en la stockant dans le cache.

=> On parle de **défaut de cache**. (b)